# MODUL P7 – SEGMENTASI CITRA DENGAN THRESHOLDING

## A. Tujuan pembelajaran

Setelah menyelesaikan praktikum ini, mahasiswa mampu:

1. Menjelaskan konsep segmentasi citra dan posisi thresholding di antara metode segmentasi lain (C2).
2. Menerapkan lima jenis global threshold, adaptive threshold, dan Otsu dengan OpenCV, serta mengimplementasikannya secara manual dengan NumPy (C3).
3. Memilih metode dan nilai threshold berdasarkan histogram dan kondisi pencahayaan citra (C4).
4. Mengevaluasi hasil segmentasi secara kuantitatif memakai IoU, Dice, dan pixel accuracy (C5).
5. Menerapkan K-Means untuk segmentasi warna dan merangkai segmentasi dengan pasca-proses morfologi dan OCR (C3).

## B. ALAT DAN BAHAN

1. PC/LAPTOP
2. Github
3. *Google Colaboratory*

## C. ULASAN TEORI

### C.1 Pengertian Segmentasi

Segmentasi citra adalah proses membagi sebuah citra digital menjadi beberapa area atau region yang memiliki kemiripan sifat visual seperti intensitas, warna, atau tekstur. Tujuannya adalah memisahkan bagian citra yang dianggap penting (objek) dari bagian yang kurang relevan (latar belakang). Hasilnya adalah citra yang terdiri dari beberapa segmen yang dapat dianalisis lebih lanjut secara terpisah.

Metode segmentasi dapat dikelompokkan menurut prinsip kerjanya. Thresholding adalah metode berbasis intensitas yang paling sederhana dan menjadi dasar banyak metode lain.

| Kelompok | Prinsip | Contoh metode |
|---|---|---|
| **Thresholding** | Memisahkan piksel dengan batas intensitas (global, lokal, otomatis). | Global, adaptive, Otsu, multi-Otsu, Triangle, Sauvola |
| **Berbasis tepi** | Mencari diskontinuitas intensitas lalu menutup kontur. | Sobel, Canny, Roberts |
| **Berbasis region** | Mengelompokkan piksel bertetangga yang homogen. | Region growing, split and merge, watershed, mean shift |
| **Clustering** | Mengelompokkan piksel di ruang fitur (warna, posisi, tekstur). | K-Means, mean shift, GMM |
| **Kontur aktif** | Kurva yang bergerak meminimalkan fungsi energi. | Snake, level set |
| **Deep learning** | Jaringan dilatih dengan citra bertanda. | U-Net, Mask R-CNN, Segment Anything Model (SAM) |

## C.2 Pengertian Operasi Thresholding

Thresholding adalah bentuk metode paling sederhana dari segmentasi citra. Biasanya digunakan pada citra grayscale / warna dan ide dasarnya adalah bagaimana memisahkan antara objek foreground dengan objek backgroundnya.

Gambar berikut menunjukkan citra asli sebelum dan sesudah di lakukan thresholding.



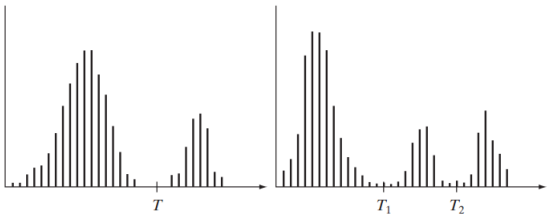
Gambar 1. Citra kiri sebelum di threshold, citra kanan setelah dithreshold (citra binary)

Beberapa metode dari Threshold yang akan dibahas pada modul ini diantaranya:

a. Global Threshold
b. Adaptive Threshold
c. Otsu's Threshold
d. Pengenalan Segmentasi Citra menggunakan K-Means

## C.3 Global Threshold

Perhatikan histogram dari sebuah image yang ditunjukkan pada gambar berikut:




Gambar 2. Intensitas histogram dapat dipisahkan dengan 1 Threshold (a) atau multiple Threshold (b)

Dianggap bahwa gambar diatas adalah histogram dari citra f(x,y), yang terdiri dari object terang pada background gelap. Dengan asumsi bahwa object dan background memiliki nilai intensitas warna yang dapat dipisahkan dalam 2 grup dominan. Cara yang termudah dan paling jelas adalah dengan memilih nilai tertentu (Threshold, *T*) yang akan memisahkan dua grup tersebut. Tiap titik (x,y) dalam image dimana f(x,y) > T dapat diistilahkan dengan *object point*, sedangkan grup satunya disebut dengan *background point*. Dengan kata lain, citra tersegmentasi g(x,y) dituliskan sebagai berikut:

$$g(x,y) = \begin{cases} 1 & jika\ f(x,y) > T \\ 0 & jika\ f(x,y) \leq T \end{cases}$$

Ketika T ditentukan secara konstanta pada keseluruhan citra, maka proses inilah yang disebut sebagai *global thresholding*. Ketika nilai T berubah-ubah dalam dalam proses satu citra, maka hal ini disebut dengan *variable/adaptive thresholding*.

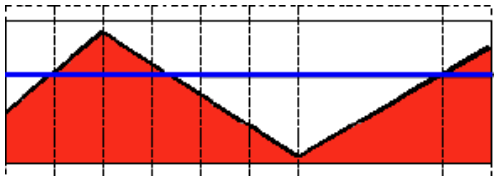
Gambar 2(a) diatas menunjukkan permasalahan thresholding yang melibatkan 3 grup dominan. Diasumsikan bahwa ada 2 grup terang dan 1 grup gelap pada histogram tersebut. Proses multiple threholding akan mengelompokkan titik (x,y) sebagai background jika f(x,y) <= $T_1$, ke grup object satu jika $T_1$ < f(x,y) <= $T_2$, dan object grup lain jika f(x,y) > $T_2$. Atau dapat dituliskan sebagai berikut:

$$g(x,y) = \begin{cases} a & jika\ f(x,y) > T_2 \\ b & jika\ T_1 < f(x,y) \leq T_2 \\ c & jika\ f(x,y) \leq T_1 \end{cases}$$

Pada OpenCV telah disediakan library untuk Thresholding, baik itu Global, Adaptive, maupun Otsu Threshold. Berikut adalah beberapat Global Thresholding yang disediakan:

a. Binary Threshold
b. Binary-Inverted Threshold
c. Truncate Threshold
d. Threshold To Zero
e. Threshold To Zero-Inverted

Untuk mengilustrasikan cara kerja masing-masing threshold, perhatikan histogram dari image asli *src(x,y)* pada gambar 3 berikut. Garis horizontal biru menunjukkan nilai threshold (tetap).



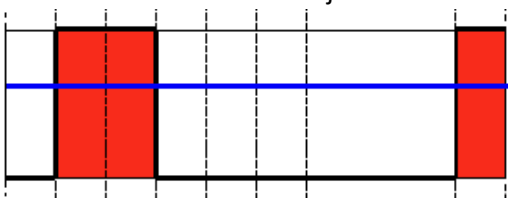
Gambar 3. Histogram dari citra *src(x,y)*

a. Binary Threshold

Binary Threshold seperti yang dijelaskan sebelumnya, merupakan threshold yang akan memisahkan dua grup warna. Untuk threshold ini, masing-masing kelompok warna akan diubah ke nilai gelap (hitam) untuk object yang dianggap background, dan diubah ke nilai terang (putih) untuk object yang dianggap foreground. Berikut adalah persamaannya:

$$dst(x,y) = \begin{cases} maxVal & jika\ src(x,y) > thresh \\ 0 & jika\ lainnya \end{cases}$$

Jadi jika nilai intensitas piksel warna pada *src(x,y)* lebih tinggi dari *thresh*, maka nilainya akan diubah menjadi *maxVal*. Selain itu akan diubah menjadi 0.



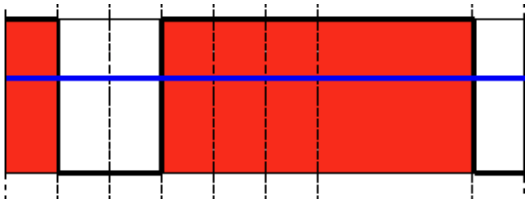
Gambar 4. Histogram hasil dari Binary Threshold

b. Binary-Inverted Threshold

Threshold ini merupakan kebalikan dari Binary Threshold. Jika intensitas warna diatas Threshold, maka nilai akan diubah menjadi 0 dan sebaliknya. Berikut adalah persamaannya:

$$dst(x,y) = \begin{cases} 0 & jika\ src(x,y) > thresh \\ maxVal & jika\ lainnya \end{cases}$$

Gambar 5 berikut merupakan histogram hasil Binary-Inverted Threshold.

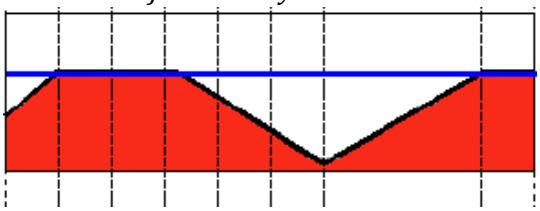


Gambar 5. Histogram hasil dari Binary-Inverted Threshold

c. Truncate Threshold

Jika nilai intensitas warna *src(x,y)* lebih besar dari *thresh*, maka nilainya akan ditruncate. Berikut adalah persamaan dan Histogram hasilnya.

$$dst(x,y) = \begin{cases} thresh & jika\ src(x,y) > thresh \\ src(x,y) & jika\ lainnya \end{cases}$$

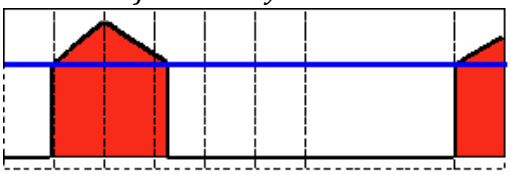


Gambar 6. Histogram hasil dari Truncate Threshold

d. Threshold To Zero

Jika nilai *src(x,y)* lebih rendah dari nilai *thresh*, maka nilai pixel barunya akan diubah menjadi 0. Berikut adalah persamaan dan gambar Histogram hasil thresholdnya.

$$dst(x,y) = \begin{cases} src(x,y) & jika\ src(x,y) > thresh \\ 0 & jika\ lainnya \end{cases}$$

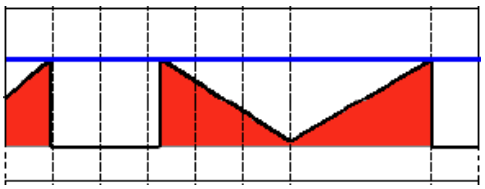


Gambar 7. Histogram hasil dari Threshold To Zero

e. Threshold To Zero – Inverted

Jika nilai *src(x,y)* lebih besar dari nilai *thresh*, maka nilai pixel barunya akan diubah menjadi 0. Berikut adalah persamaan dan gambar Histogram hasil thresholdnya.

$$dst(x,y) = \begin{cases} 0 & jika\ src(x,y) > thresh \\ src(x,y) & jika\ lainnya \end{cases}$$



Gambar 8. Histogram hasil dari Threshold To Zero

Global threshold yang telah disebutkan diatas dapat digunakan secara langsung menggunakan library OpenCV cv.Threshold. berikut adalah link dokumentasinya:

https://docs.opencv.org/4.4.0/d7/d4d/tutorial_py_thresholding.html

Code Berikut menunjukkan penggunaan dari library OpenCV cv.Threshold

```
import cv2 as cv
from google.colab.patches import cv2_imshow
import numpy as np
import matplotlib.pyplot as plt

filename = ('/content/drive/MyDrive/PCVK/Images/gradient.jpg')
img = cv.imread(filename)
thresh = 127      #nilai Threshold yang ditentukan
maxval=255

#1. threshold binary jika pixel di img>127, maka thresh1 bernilai 1(putih)
selain itu bernilai 0(hitam)
ret,thresh1 = cv.threshold(img,thresh,maxval,cv.THRESH_BINARY)

#2. threshold binary inverse jika pixel>thresh, maka pixel bernilai 0; selain
itu akan bernilai maxval
ret,thresh2 = cv.threshold(img,thresh,maxval,cv.THRESH_BINARY_INV)

#3. Threshold Truncate jika pixel>thresh, maka pixel bernilai thresh; selain
itu akan bernilai tetap
ret,thresh3 = cv.threshold(img,thresh,maxval,cv.THRESH_TRUNC)

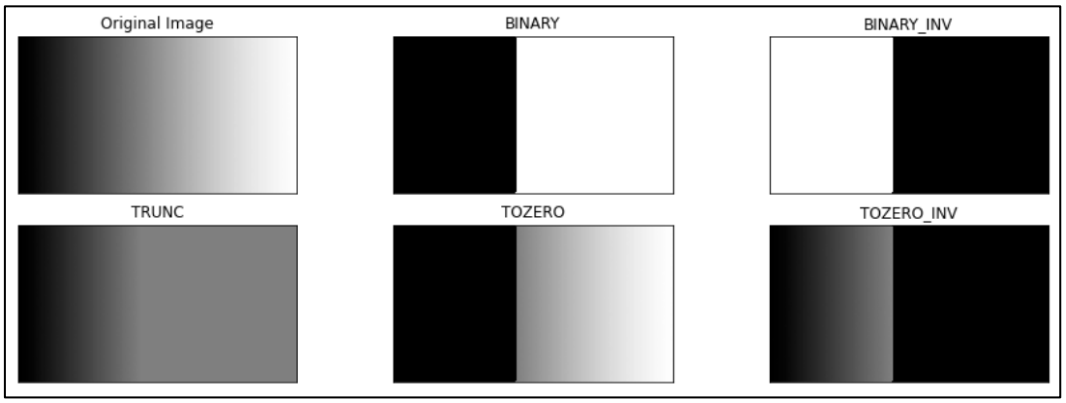
#4. Threshold Tozero jika pixel>thresh, maka pixel bernilai tetap; selain itu
akan bernilai 0
ret,thresh4 = cv.threshold(img,thresh,maxval,cv.THRESH_TOZERO)

#5. Threshold Tozero Inverse jika pixel>thresh, maka pixel bernilai 0; selain
itu akan bernilai tetap
ret,thresh5 = cv.threshold(img,thresh,maxval,cv.THRESH_TOZERO_INV)

titles = ['Original Image','BINARY','BINARY_INV','TRUNC', 'TOZERO',
'TOZERO_INV']
images = [img, thresh1, thresh2, thresh3, thresh4, thresh5]

plt.figure(figsize = (15,5))
for i in range(len(images)):
  plt.subplot(2,3,i+1),plt.imshow(images[i],'gray', interpolation='nearest')
  plt.title(titles[i])
  plt.xticks([]),plt.yticks([])
plt.show()
```

Gambar berikut menunjukkan perbedaan masing-masing global threshold yang merupakan keluaran dari code diatas. Seperti yang terlihat pada code tersebut, nilai Global thresholdnya adalah 127 dan nilai tersebut berlaku untuk keseluruhan image.




Gambar 9. Global Threshold dari Library OpenCV

## C.4 Adaptive Threshold

Pada bagian sebelumnya kita menggunakan global thresholding, yaitu satu nilai ambang T untuk seluruh citra. Pendekatan ini bekerja baik jika pencahayaan merata dan histogram memperlihatkan pemisahan yang jelas antara objek dan latar. Kita juga telah melihat bahwa ia dapat gagal: pada citra Lena low contrast (intensitas 85-108), threshold tetap 127 menghasilkan citra hitam seluruhnya. Otsu memperbaiki masalah tersebut dengan menghitung T dari histogram, tetapi hasilnya tetap satu nilai untuk seluruh citra.

Masalah lain muncul ketika pencahayaan tidak merata, misalnya foto dokumen yang separuhnya terkena bayangan, atau papan sudoku yang difoto dari samping dengan satu sisi lebih gelap. Pada kondisi ini tidak ada satu nilai T yang cocok untuk semua bagian: T yang tepat di area terang akan menghapus tulisan di area gelap, dan sebaliknya.

Adaptive threshold (thresholding adaptif atau lokal) menjawab masalah itu dengan satu gagasan sederhana: setiap piksel mempunyai ambangnya sendiri, yang dihitung dari lingkungan di sekitar piksel tersebut.

### Konsep Dasar

Pada global threshold, keputusan untuk piksel I(x, y) adalah:

$$g(x,y) = \begin{cases} 255 & I(x,y) > T \\ 0 & \text{selain itu} \end{cases}$$

Pada adaptive threshold, *T* bukan lagi konstanta, melainkan fungsi posisi:

$$g(x,y) = \begin{cases} 255 & I(x,y) > T(x,y) \\ 0 & \text{selain itu} \end{cases}$$

Nilai *T(x, y)* dihitung dari **jendela (window) berukuran *b* × *b*** yang berpusat di piksel (x, y). Piksel dibandingkan dengan keadaan lokalnya, bukan dengan nilai global. Dengan demikian, piksel yang lebih terang daripada lingkungannya menjadi putih, dan yang lebih gelap menjadi hitam, **walaupun nilai absolutnya berbeda di bagian citra yang berlainan**.

### Cara Menghitung Ambang Lokal

Beberapa cara umum menentukan *T(x, y)*:

| Metode | Rumus *T(x, y)* | Keterangan |
|---|---|---|
| **Rata-rata lokal** (mean) | $\mu(x, y) - C$ | Rata-rata jendela dikurangi konstanta *C* |
| **Gaussian lokal** | $\mu_G(x, y) - C$ | Rata-rata berbobot Gaussian; piksel dekat pusat berpengaruh lebih besar |
| **Niblack** | $\mu + k \cdot \sigma$ | Menambahkan simpangan baku lokal σ; *k* biasanya negatif |
| **Sauvola** | $\mu\,[1 + k\,(\sigma / R - 1)]$ | Perbaikan Niblack untuk dokumen; *R* adalah rentang maksimum simpangan baku (umumnya 128), *k* sekitar 0,2 sampai 0,5 |

Praktikum ini memakai dua yang pertama (ADAPTIVE_THRESH_MEAN_C dan ADAPTIVE_THRESH_GAUSSIAN_C pada OpenCV). Niblack dan Sauvola tersedia di skimage.filters sebagai pengayaan. Di OpenCV seluruh proses ini dikerjakan oleh satu fungsi:
`cv.adaptiveThreshold(citra, 255, metode, jenis, blockSize, C).`

**Dua parameter utama**

1. **Ukuran blok (*block size*, *b*)**
   Ukuran jendela, harus ganjil (misalnya 11, 31, 51). Ukuran ini menentukan seberapa "lokal" keputusannya:
   - *b* terlalu kecil: jendela hanya mencakup sedikit piksel, sehingga *T* mengikuti noise dan tekstur halus. Hasilnya berbintik, dan bagian dalam objek yang besar bisa berlubang.
   - *b* terlalu besar: perilakunya mendekati global threshold, sehingga keunggulan adaptifnya hilang.
   - Aturan praktis: *b* sedikit lebih besar daripada objek terbesar yang ingin dipertahankan (misalnya lebih besar dari lebar goresan huruf).
2. **Konstanta *C***
   Nilai yang dikurangkan dari rata-rata lokal. Fungsinya menentukan seberapa jauh piksel harus lebih terang daripada rata-rata lingkungannya agar dianggap putih:
   - *C* = 0: piksel sedikit di atas rata-rata langsung menjadi putih, sehingga **area yang homogen (misalnya latar polos) penuh bintik noise** karena nilainya berfluktuasi di sekitar rata-ratanya sendiri.
   - *C* > 0: ambang diturunkan, sehingga area homogen menjadi bersih (putih), dan hanya piksel yang cukup gelap yang dianggap objek.
   - *C* terlalu besar: objek yang kontrasnya tipis ikut hilang.

Untuk citra *low contrast*, *C* perlu dipilih kecil. Misalnya pada citra Lena yang rentangnya hanya 24 level, *C* sebesar 20 sudah lebih besar daripada seluruh rentang intensitasnya.

**Langkah Algoritma**

1. Ubah citra ke grayscale (dan, bila perlu, haluskan dengan Gaussian blur untuk mengurangi noise).
2. Pilih ukuran blok *b* (ganjil) dan konstanta *C*.
3. Untuk setiap piksel (x, y), hitung rata-rata (atau rata-rata Gaussian) intensitas pada jendela $b \times b$ di sekitarnya.
4. Hitung ambang lokal *T(x, y) = rata-rata − C*.
5. Bandingkan: jika *I(x, y) > T(x, y)* maka piksel menjadi putih, selain itu hitam.
6. Ulangi untuk seluruh piksel menghasilkan citra biner.

Berikut adalah contoh code untuk Adaptive Threshold:

```
filename = ('/content/drive/MyDrive/PCVK/Images/sudoku-original.jpg')
citra = cv.medianBlur(cv.imread(filename),5)
gray = cv.cvtColor(citra, cv.COLOR_BGR2GRAY)

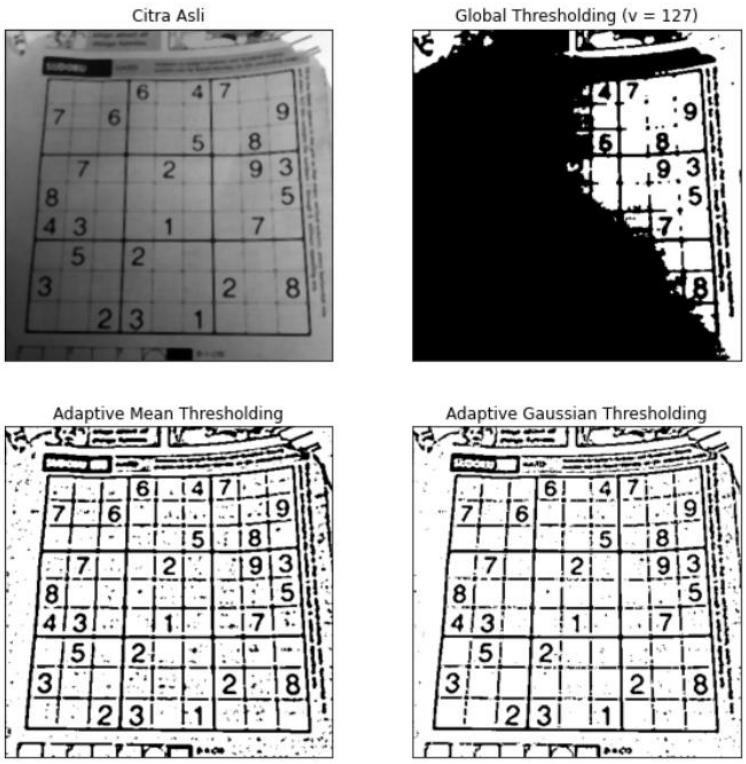
thresh = 127

ret,th1 = cv.threshold(gray,thresh,255,cv.THRESH_BINARY)
th2 = cv.adaptiveThreshold(gray,255,cv.ADAPTIVE_THRESH_MEAN_C,
cv.THRESH_BINARY,11,2)
th3 = cv.adaptiveThreshold(gray,255,cv.ADAPTIVE_THRESH_GAUSSIAN_C,
cv.THRESH_BINARY,11,2)

titles = ['Citra Asli', 'Global Thresholding (v = 127)',
            'Adaptive Mean Thresholding', 'Adaptive Gaussian Thresholding']
citra2 = [gray, th1, th2, th3]

plt.figure(figsize = (10,10))
for i in range(len(citra2)):
    plt.subplot(2,2,i+1),plt.imshow(citra2[i],'gray')
    plt.title(titles[i])
    plt.xticks([]),plt.yticks([])
plt.show()
```

Berikut adalah gambar hasil dari code diatas.




Gambar 10. Citra hasil Adaptive Threshold Mean dan Gaussian Window

**Kelebihan dan Keterbatasan**

| Aspek | Keterangan |
|---|---|
| **Kelebihan** | Tahan terhadap pencahayaan tidak merata dan bayangan; tidak memerlukan histogram yang bimodal; cocok untuk dokumen, teks, dan papan bergaris seperti sudoku; relatif cepat |
| **Keterbatasan** | Dua parameter (*b* dan *C*) harus disetel; area homogen menghasilkan noise bila *C* terlalu kecil; objek besar dan seragam dapat berlubang di bagian dalam karena bagian tengahnya mirip rata-rata lingkungannya sendiri; hasil bergantung pada skala objek |

## C.5 Otsu's Threshold

Otsu's threshold adalah metode segmentasi otomatis yang sangat populer, bahkan sampai hari ini pun masih sering digunakan sebagai standar metode binary threshold yang handal. Threshold ini dinamakan sesuai dengan nama penemunya yaitu Nobuyuki Otsu. Dalam bentuk yang paling sederhana, algoritma ini akan melakukan segmentasi pixel menjadi dua kelas, yaitu foreground dan background. Threshold ini ditentukan dengan meminimasikan intensitas variance intra-class, atau bisa juga dengan memaksimalkan variance inter-class.

Algoritma Otsu's secara iteratif mencari threshold yang terkecil dari variance intra-class, yang didefinisikan sebagai jumlah bobot dari variance dua class:

$$\sigma_w^2(t) = \omega_0(t)\sigma_0^2(t) + \omega_1(t)\sigma_1^2(t)$$

Bobot $\omega_0$ dan $\omega_1$adalah probabilitas dari 2 class yang dipisahkan oleh threshold $t$. $\sigma_0^2$dan $\sigma_1^2$adalah variance dari masing-masing class.

$$\omega_0(t) = \sum_{i=0}^{t-1} p(i)$$

$$\omega_1(t) = \sum_{i=t}^{L-1} p(i)$$

Mencari nilai minimal dari variance intra-class ekivalen dengan mencari nilai maksimal dari variance inter-class.

$$\sigma_B^2(t) = \sigma^2 - \sigma_W^2(t) = \omega_0(\mu_0 - \mu_T)^2 + \omega_1(\mu_1 - \mu_T)^2$$
$$= \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2$$

Dimana nilai mean class $\mu_0(t)$, $\mu_1(t)$, dan $\mu_T$ adalah:

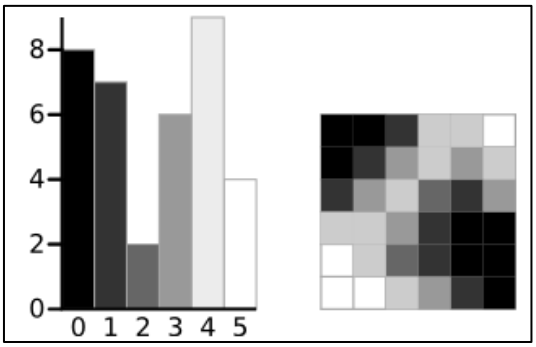
$$\mu_0(t) = \frac{\sum_{i=0}^{t-1} ip(i)}{\omega_0(t)}$$

$$\mu_1(t) = \frac{\sum_{i=t}^{L-1} ip(i)}{\omega_1(t)}$$

$$\mu_T = \sum_{i=0}^{L-1} ip(i)$$

Perhatikan contoh perhitungan algoritma Otsu's berikut:

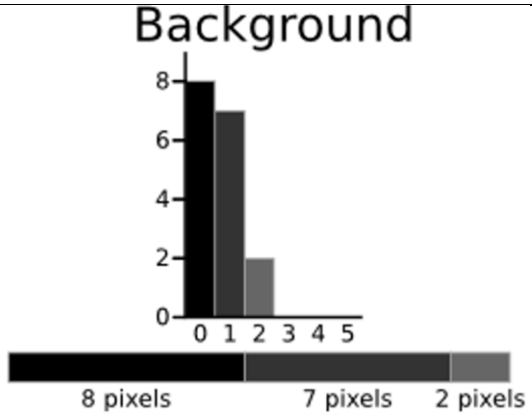
Gambar berikut menunjukkan sebuah image resolusi 6x6 dan terdiri dari 6 warna, beserta histogramnya.




Gambar 11. Contoh citra resolusi 6x6 beserta histogramnya

Dari gambar diatas, lakukan perhitungan Weight, Mean, Variance untuk tiap background dan foreground untuk tiap nilai threshold. Contoh berikut adalah perhitungan dengan nilai threshold = 3.

Threshold = 3




Gambar 12. Histogram untuk background object dengan T=3

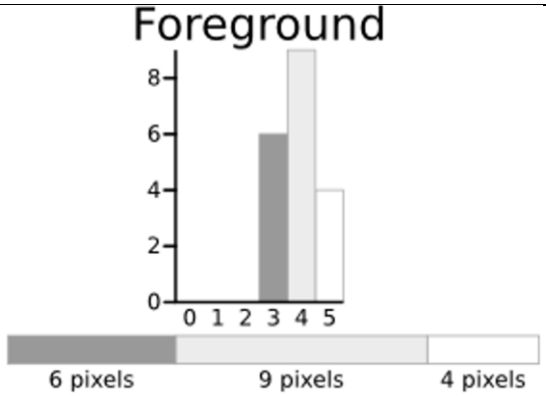
weight: $\omega_b = \frac{8+7+2}{36} = 0.4722$

mean $\mu_b = \frac{(0\ x\ 8)+(1\ x\ 7)+(2\ x\ 2)}{17} = 0.6471$

variance

$$\sigma_b^2 = \frac{((0-0.6471)^2 x8) + ((1-0.6471)^2 x7) + ((2-0.6471)^2 x2)}{17}$$

$$= \frac{(0.4187x8) + (0.1246x7) + (1.8304x2)}{17} = \mathbf{0.4637}$$




Gambar 13. Histogram untuk Foreground object dengan T=3

weight: $\omega_f = \frac{6+9+4}{36} = 0.5278$

mean $\mu_f = \frac{(3\ x\ 6)+(4\ x\ 9)+(5\ x\ 4)}{19} = 3.8947$

variance

$$\sigma_f^2 = \frac{((3-3.8947)^2 x6) + ((4-3.8947)^2 x9) + ((5-3.8947)^2 x4)}{19}$$

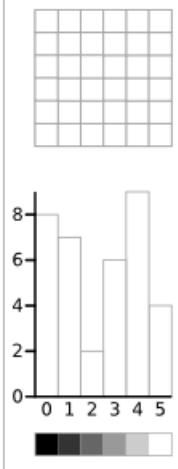
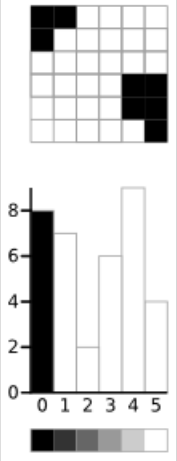
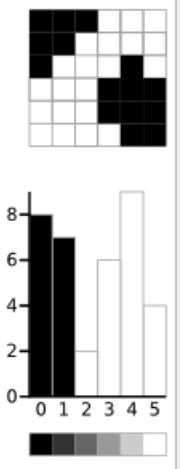
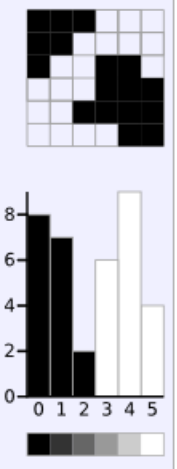
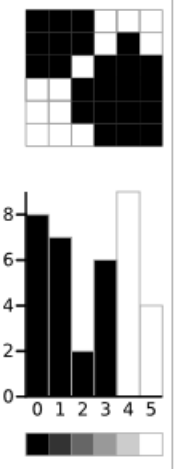
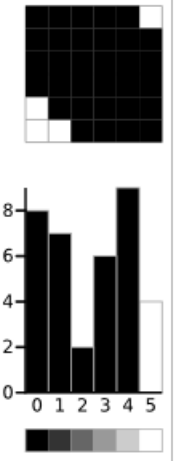
$$= \frac{(0{,}80048x6) + (0{,}011088x9) + (1{,}22168x4)}{19} = \mathbf{0.5152}$$

Selanjutnya nilai within class variance dari 2 variance (background dan foreground) dapat dihitung dengan :

Within class variance:

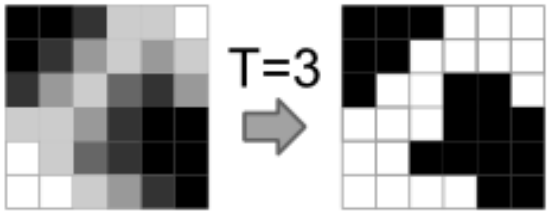
$\sigma_W^2 = \omega_b \sigma_b^2 + \omega_f \sigma_f^2 = 0.4722\ x\ 0.4637 + 0.5278\ x\ 0.5152 = \mathbf{0.4909}$

Perhitungan ini dilakukan untuk setiap nilai threshold yang mungkin (0 <= T <= 5). Nilai Within class variance **terkecil** akan menjadi threshold yang terpilih.

| Threshold | T=0 | T=1 | T=2 | T=3 | T=4 | T=5 |
|---|---|---|---|---|---|---|
| |  |  |  |  |  |  |
| Weight, Background | $W_b$ = 0 | $W_b$ = 0.222 | $W_b$ = 0.4167 | $W_b$ = 0.4722 | $W_b$ = 0.6389 | $W_b$ = 0.8889 |
| Mean, Background | $M_b$ = 0 | $M_b$ = 0 | $M_b$ = 0.4667 | $M_b$ = 0.6471 | $M_b$ = 1.2609 | $M_b$ = 2.0313 |
| Variance, Background | $\sigma^2_b$ = 0 | $\sigma^2_b$ = 0 | $\sigma^2_b$ = 0.2489 | $\sigma^2_b$ = 0.4637 | $\sigma^2_b$ = 1.4102 | $\sigma^2_b$ = 2.5303 |
| Weight, Foreground | $W_f$ = 1 | $W_f$ = 0.7778 | $W_f$ = 0.5833 | $W_f$ = 0.5278 | $W_f$ = 0.3611 | $W_f$ = 0.1111 |
| Mean, Foreground | $M_f$ = 2.3611 | $M_f$ = 3.0357 | $M_f$ = 3.7143 | $M_f$ = 3.8947 | $M_f$ = 4.3077 | $M_f$ = 5.000 |
| Variance, Foreground | $\sigma^2_f$ = 3.1196 | $\sigma^2_f$ = 1.9639 | $\sigma^2_f$ = 0.7755 | $\sigma^2_f$ = 0.5152 | $\sigma^2_f$ = 0.2130 | $\sigma^2_f$ = 0 |
| Within Class Variance | $\sigma^2_W$ = 3.1196 | $\sigma^2_W$ = 1.5268 | $\sigma^2_W$ = 0.5561 | $\sigma^2_W$ = **0.4909** | $\sigma^2_W$ = 0.9779 | $\sigma^2_W$ = 2.2491 |

Gambar 14. Perhitungan Within Class Variance untuk seluruh nilai T yang mungkin

Gambar berikut menunjukkan hasil citra yang dithreshold dengan Otsu (T=3)




Gambar 15. Citra Threshold hasil Otsu's dengan nilai T=3

Penentuan nilai threshold yang terpilih juga bisa dilakukan menggunakan perhitungan Between class variance, dimana nilai Between class variance **terbesar** adalah threshold yang terpilih.

Between class variance:

$$\sigma_B^2 = \sigma^2 - \sigma_W^2$$
$$= \omega_b(\mu_b - \mu)^2 + \omega_f(\mu_f - \mu)^2 \qquad (dimana\ \mu = \omega_b\mu_b + \omega_f\mu_f)$$
$$= \omega_b\omega_f(\mu_b - \mu_f)^2$$

| Threshold | T=0 | T=1 | T=2 | T=3 | T=4 | T=5 |
|---|---|---|---|---|---|---|
| Within Class Variance | $\sigma^2_W$ = 3.1196 | $\sigma^2_W$ = 1.5268 | $\sigma^2_W$ = 0.5561 | **$\sigma^2_W$ = 0.4909** | $\sigma^2_W$ = 0.9779 | $\sigma^2_W$ = 2.2491 |
| Between Class Variance | $\sigma^2_B$ = 0 | $\sigma^2_B$ = 1.5928 | $\sigma^2_B$ = 2.5635 | **$\sigma^2_B$ = 2.6287** | $\sigma^2_B$ = 2.1417 | $\sigma^2_B$ = 0.8705 |

Otsu's telah disediakan pula dalam library OpenCV (cv.THRESH_OTSU) yang biasanya dikombinasikan dengan binary threshold(cv.THRESH_BINARY). Library ini dipanggil dengan object cv.threshold. Berikut adalah code contoh penggunaan Otsu's dengan library OpenCV.

```
# Dengan Library

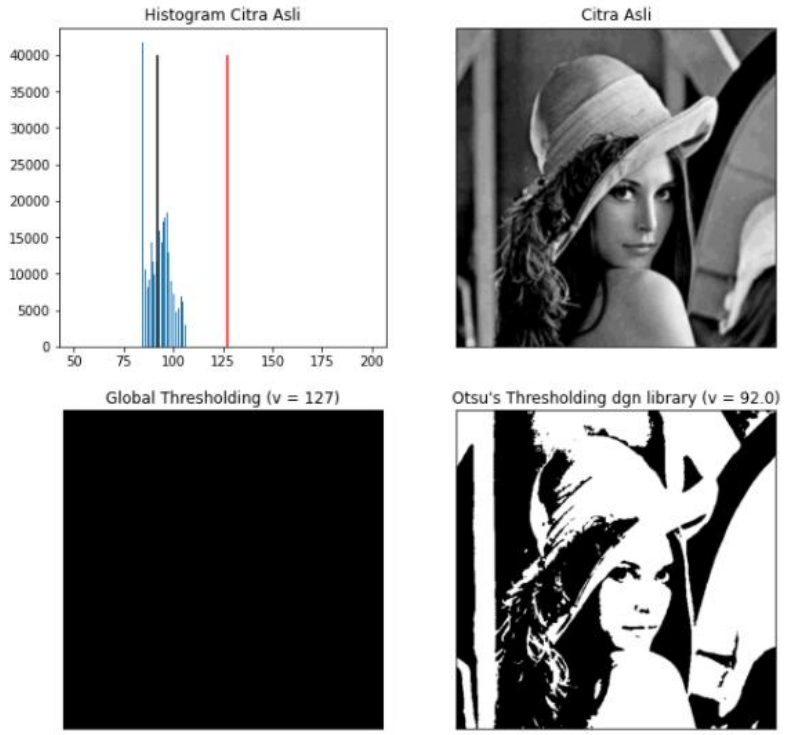
filename = ('/content/drive/MyDrive/PCVK/Images/lena_gs_lc2.jpg')
img = cv.imread(filename,0)
blur = cv.GaussianBlur(img,(5,5),0)
thresh = 127

ret,th1 = cv.threshold(blur,thresh,255,cv.THRESH_BINARY)
ret2,th2 = cv.threshold(blur,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)

x = ("Otsu's Thresholding dgn library (v = ")+str(ret2)+")"
titles = ['Citra Asli', 'Global Thresholding (v = 127)', x]
citra3 = [blur, th1, th2]

plt.figure(figsize = (10,10))
plt.subplot(2,2,1),plt.hist(blur.ravel(),256,[50,200])
plt.vlines(ret,0,40000,colors='red') #garis vertikal merah menunjukan threshold
global 127
plt.vlines(ret2,0,40000,colors='black') #garis vertikal hitam menunjukkan
threshold 92 hasil otsu's
plt.title('Histogram Citra Asli')
for i in range(len(citra3)):
 plt.subplot(2,2,i+2),plt.imshow(citra3[i],'gray')
 plt.title(titles[i])
 plt.xticks([]),plt.yticks([])
plt.show()
```

Gambar berikut menunjukkan hasil dari code diatas. Garis vertikal merah pada histogram menunjukkan nilai threshold 127. Dari gambar tersebut terlihat bahwa warna disebelah kiri garis merah akan diubah semua ke warna hitam, sehingga hasil thresholdnya akan menjadi warna hitam semua. Berbeda dengan hasil dari otsu's yang masih dapat memisahkan dua class background dan foreground. Garis vertikal hitam menunjukkan nilai threshold yang dihasilkan oleh otsu's. terlihat bahwa garis tersebut masih bisa membagi histogram menjadi dua bagian yaitu background dan foreground.



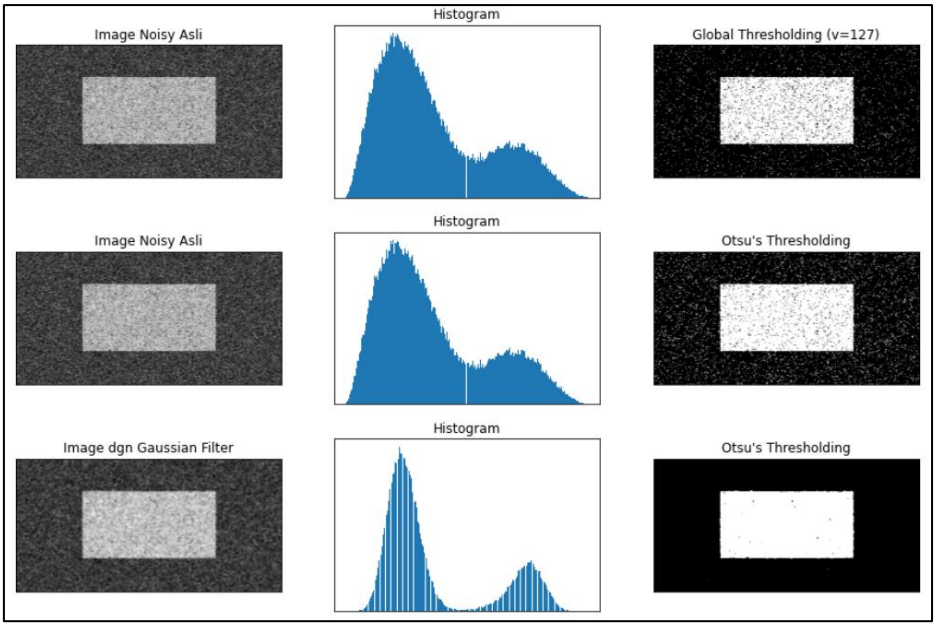

Gambar 16. Citra hasil threshold menggunakan binary threshold dan Otsu's

Code berikut akan menampilkan hasil dari threshold Otsu's tanpa Gaussian Filter dan dengan menggunakan Gaussian Filter.

```
filename = ('/content/drive/MyDrive/PCVK/Images/noisy2.png')
img = cv.imread(filename,0)
#Global Thresholding
ret1,th1 = cv.threshold(img,127,255,cv.THRESH_BINARY)
# Otsu's thresholding
ret2,th2 = cv.threshold(img,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)
# Otsu's thresholding setelah dilakukan Gaussian filtering
blur = cv.GaussianBlur(img,(5,5),0)
ret3,th3 = cv.threshold(blur,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)
#plotting semua image
images = [img, 0, th1,
          img, 0, th2,
          blur, 0, th3]
titles = ['Image Noisy Asli','Histogram','Global Thresholding (v=127)',
          'Image Noisy Asli','Histogram',"Otsu's Thresholding",
          'Image dengan Gaussian Filter','Histogram',"Otsu's Thresholding"]

plt.figure(figsize = (15,10))
for i in range(3):
    plt.subplot(3,3,i*3+1),plt.imshow(images[i*3],'gray')
    plt.title(titles[i*3]), plt.xticks([]), plt.yticks([])
    plt.subplot(3,3,i*3+2),plt.hist(images[i*3].ravel(),256)
    plt.title(titles[i*3+1]), plt.xticks([]), plt.yticks([])
    plt.subplot(3,3,i*3+3),plt.imshow(images[i*3+2],'gray')
    plt.title(titles[i*3+2]), plt.xticks([]), plt.yticks([])
plt.show()
```

Gambar berikut menunjukkan hasil dari code diatas. Dapat disimpulkan bahwa dengan melakukan filtering terlebih dahulu terhadap citra masukan, hasil threshold Otsu's dapat lebih baik.




Gambar 17. Citra Otsu's tanpa preprocess dan dengan preprocess Gaussian Filter

## C.6 Segmentasi citra berbasis wilayah (Region-Based Segmentation)

Pada bagian sebelumnya, kita telah mempelajari segmentasi dengan thresholding, yaitu memisahkan objek dan latar belakang berdasarkan satu nilai ambang. Pendekatan tersebut bekerja dengan baik jika histogram citra memperlihatkan pemisahan yang jelas antara objek dan latar. Namun, kita juga telah melihat bahwa thresholding bisa gagal ketika kontras citra rendah, misalnya pada citra Lena dengan rentang intensitas 85-108, di mana threshold tetap 127 menghasilkan citra hitam seluruhnya.

Segmentasi berbasis wilayah menawarkan cara pandang yang berbeda. Alih-alih bertanya "apakah nilai piksel ini di atas atau di bawah ambang?", metode ini bertanya "apakah piksel ini mirip dengan tetangga-tetangganya?". Dengan demikian, citra dibagi menjadi beberapa wilayah (region) yang masing-masing terdiri dari piksel-piksel yang saling berdekatan dan memiliki karakteristik serupa.

**Konsep Dasar**

Secara formal, segmentasi citra I menjadi n region $R_1$, $R_2$, ..., $R_n$ harus memenuhi syarat berikut:

1. Lengkap: gabungan seluruh region membentuk citra utuh, sehingga setiap piksel termasuk dalam satu region.
2. Terhubung: setiap region berupa kumpulan piksel yang saling bersebelahan.
3. Saling lepas: tidak ada piksel yang menjadi anggota dua region sekaligus.
4. Homogen: setiap region memenuhi predikat kesamaan $P(R_i)$ = TRUE.
5. Berbeda antar region: gabungan dua region yang bertetangga tidak memenuhi predikat kesamaan, yaitu $P(R_i \cup R_j)$ = FALSE.

Syarat ke-4 dan ke-5 inilah inti dari pendekatan berbasis wilayah. Region harus homogen di dalam dan berbeda dengan region di sekitarnya.

**Ciri-ciri pendekatan ini**

- Fokus pada kesamaan antar piksel tetangga. Keputusan pengelompokan didasarkan pada hubungan lokal antar piksel yang berdekatan, bukan pada nilai absolut piksel terhadap suatu ambang global.
- Menekankan keterhubungan area dibanding batas tepi. Metode berbasis tepi (edge-based) mencari tempat intensitas berubah drastis, sedangkan metode berbasis wilayah mencari area yang konsisten. Keduanya saling melengkapi.
- Biasanya diawali dari titik awal (seed point). Pada banyak metode, proses dimulai dari satu atau beberapa piksel acuan yang kemudian meluas ke piksel di sekitarnya selama kriteria kesamaan terpenuhi.
- Cocok untuk citra yang tidak memiliki kontras tegas namun area objek cukup homogen. Karena yang dinilai adalah kemiripan lokal, metode ini lebih tahan terhadap citra berkontras rendah dibandingkan threshold global.

**Kriteria Kesamaan (Homogeneity Criteria)**

Kriteria kesamaan adalah aturan yang menentukan apakah suatu piksel (atau region) boleh digabungkan ke dalam region yang sudah ada. Suatu piksel dianggap sesuai jika memenuhi salah satu kondisi berikut:

| **Kriteria** | **Penjelasan** | **Contoh Penerapan** |
|---|---|---|
| Perbedaan intensitas | Selisih nilai keabuan antara piksel kandidat dan region lebih kecil dari ambang *T*: \|I(q) − μ_region\| < T | Region Growing pada citra grayscale |
| Perbedaan warna | Jarak warna di ruang RGB atau HSV lebih kecil dari batas tertentu. HSV sering lebih baik karena memisahkan warna (Hue) dari kecerahan (Value) | Segmentasi objek berwarna pada citra berwarna |
| Variansi intensitas | Variansi nilai piksel di dalam region tetap di bawah nilai tertentu, artinya region tidak terlalu "beragam" | Predikat pada Split & Merge |
| Keseragaman tekstur | Pola tekstur lokal seragam, misalnya nilai *energy* GLCM (Gray-Level Co-occurrence Matrix) yang tinggi menandakan tekstur teratur | Segmentasi area berpola, seperti kain atau rumput |

Pemilihan kriteria sangat memengaruhi hasil. Ambang yang terlalu ketat menghasilkan banyak region kecil (*over-segmentation*), sedangkan ambang yang terlalu longgar membuat objek berbeda menyatu menjadi satu region (*under-segmentation*).

**Jenis Metode Segmentasi Berbasis Wilayah**

Praktikum ini membahas empat metode utama. Keempatnya memiliki tujuan yang sama, tetapi berbeda dalam cara membentuk region.

**1. Region Growing**

Region dimulai dari satu atau beberapa **seed point** di dalam objek, lalu meluas ke tetangga (4 atau 8 ketetanggaan) yang intensitasnya mirip dengan region. Pertumbuhan berhenti saat tidak ada tetangga yang memenuhi kriteria. Hasilnya selalu terhubung, tetapi sangat bergantung pada posisi seed dan threshold (seed pada piksel noise bisa menggagalkan proses).

**Contoh hasil** (citra coins, seed di tengah satu koin, T = 20): mask biner hanya menutupi **satu koin yang dipilih**, sedangkan koin lain dan latar tetap hitam. Jika T terlalu kecil, region hanya berupa bercak di sekitar seed. Jika T terlalu besar, region "bocor" ke latar atau koin di sebelahnya.

**2. Region Splitting and Merging**

Citra dibagi rekursif menjadi empat kuadran (*quadtree*) sampai tiap blok homogen (*splitting*), lalu blok bertetangga yang mirip digabung kembali (*merging*). Metode ini otomatis tanpa seed, tetapi batas region cenderung **kotak-kotak**.

**Contoh hasil** (citra page atau text): area kosong dan latar polos terwakili **blok besar**, sedangkan area huruf dan tepi terpecah menjadi **blok kecil rapat**. Setelah merging, blok latar menyatu menjadi satu region luas, tetapi tepinya tetap bergerigi mengikuti pola persegi.

**3. Watershed Segmentation**

Citra (gradien atau distance transform) dianalogikan sebagai permukaan berbukit yang "dibanjiri" dari marker. Tempat air dari marker berbeda bertemu menjadi **garis batas**. Metode ini unggul memisahkan objek bersentuhan, tetapi tanpa marker terkontrol terjadi over-segmentation.

**Contoh hasil** (citra coins, Otsu, distance transform, marker): setiap koin mendapat **label warna sendiri**, dan koin yang saling bersentuhan **terpisah oleh garis batas tipis**. Tanpa marker, hasilnya berupa ratusan kepingan kecil di dalam satu koin akibat noise dan tekstur.

**4. Mean Shift Segmentation**

Setiap piksel dipandang sebagai titik di ruang fitur (x, y, r, g, b), lalu digeser ke arah rata-rata tetangganya sampai konvergen ke **mode** (puncak densitas). Piksel dengan mode sama menjadi satu region. Jumlah region tidak perlu ditentukan di awal dan tepi relatif terjaga, tetapi komputasinya berat dan butuh penyetelan bandwidth.

**Contoh hasil** (citra astronaut): gambar tampak seperti **lukisan atau kartun**. Area kulit, baju, dan latar menjadi **bidang warna rata**, sementara tepi objek utama tetap tajam. Bandwidth kecil menghasilkan banyak region halus, dan bandwidth besar menghasilkan sedikit region yang lebih "datar".

| Metode | Prinsip Utama | Perlu Seed/Marker | Kelebihan | Kekurangan |
|---|---|---|---|---|
| Region Growing | Menumbuhkan region dari seed berdasarkan kemiripan | Ya (seed) | Sederhana, region terhubung | Sensitif terhadap seed dan threshold |
| Split & Merge | Bagi (quadtree) lalu gabung | Tidak | Otomatis, tidak butuh seed | Batas region bisa kotak-kotak |
| Watershed | Citra dianalogikan sebagai topografi lalu "banjir" | Ya (marker) | Baik memisahkan objek bersentuhan | Over-segmentation tanpa marker |
| Mean Shift | Mencari puncak densitas di ruang fitur | Tidak | Jumlah cluster otomatis, tepi terjaga | Komputasi berat |

## C.7 Sementasi Citra menggunakan K-Means

Pada bagian C.6, kita telah mempelajari segmentasi berbasis wilayah yang mengelompokkan piksel berdasarkan kedekatan posisi (keterhubungan) sekaligus kemiripan nilai. Region Growing, misalnya, hanya menambahkan piksel yang bersebelahan dengan region.

Segmentasi dengan clustering memakai sudut pandang lain. Citra diperlakukan sebagai kumpulan data, di mana setiap piksel adalah satu titik dengan fitur tertentu (intensitas atau warna). Piksel-piksel dengan fitur serupa dikelompokkan ke dalam cluster yang sama, tanpa mempertimbangkan letaknya di citra. Metode yang paling populer untuk tujuan ini adalah K-Means. K-Means termasuk pembelajaran tak terawasi (unsupervised learning): algoritma tidak membutuhkan contoh berlabel, cukup jumlah kelompok K yang diinginkan.

### Konsep Dasar

Misalkan citra memiliki N piksel, dan setiap piksel $x_i$ direpresentasikan sebagai vektor fitur. Tujuan K-Means adalah membagi N piksel menjadi K cluster $C_1$, $C_2$, ..., C_K dengan pusat (centroid) $\mu_1$, $\mu_2$, ..., μ_K sehingga jumlah kuadrat jarak piksel ke centroid cluster-nya minimum:

$$J = \sum_{i=1}^{k} \sum_{x_j \in C_i} ||x_j - \mu_i||^2$$

Nilai *J* sering disebut **inertia** atau *within-cluster sum of squares* (WCSS). Semakin kecil *J*, semakin rapat piksel di dalam tiap cluster.

### Langkah Algoritma

1. **Tentukan K**, yaitu jumlah cluster yang diinginkan.
2. **Inisialisasi** K centroid, secara acak atau dengan *k-means++* agar penyebarannya lebih baik.
3. **Penugasan**: setiap piksel dimasukkan ke cluster dengan centroid terdekat.
4. **Pembaruan**: hitung ulang tiap centroid sebagai rata-rata seluruh piksel anggotanya.
5. **Ulangi** langkah 3 dan 4 sampai centroid tidak berubah (konvergen) atau iterasi maksimum tercapai.
6. **Bentuk citra hasil**: ganti nilai setiap piksel dengan nilai centroid cluster-nya, atau tampilkan label cluster sebagai peta segmentasi.

Pada tahap 6, citra hasil hanya memiliki *K* warna. Karena itu, K-Means juga dikenal sebagai teknik **kuantisasi warna** (*color quantization*).

### Memilih Nilai K

*K* adalah parameter terpenting dan harus ditentukan oleh pengguna. Beberapa pendekatan yang umum:

- **Pengetahuan tentang citra.** Jika kita tahu citra terdiri dari 3 jenis objek (misalnya langit, bangunan, dan rumput), pilih K = 3.
- **Metode Elbow.** Hitung inertia untuk beberapa nilai K, lalu cari titik "siku" pada grafik, yaitu K di mana penurunan inertia mulai melandai.

- **Silhouette score.** Mengukur seberapa mirip suatu piksel dengan cluster-nya dibandingkan cluster lain. Nilai mendekati 1 menunjukkan cluster yang baik.

Pemilihan K yang terlalu kecil menyatukan objek yang berbeda (*under-segmentation*), sedangkan K yang terlalu besar memecah satu objek menjadi banyak bagian (*over-segmentation*).

**Kelebihan dan Keterbatasan**

| **Aspek** | **Keterangan** |
|---|---|
| **Kelebihan** | Sederhana dan mudah diimplementasikan; cepat dibanding Mean Shift; tidak memerlukan seed atau marker; cocok untuk citra yang warnanya terbagi dalam kelompok yang jelas |
| **Keterbatasan** | K harus ditentukan di awal; hasil bergantung pada inisialisasi centroid (bisa terjebak di minimum lokal); cenderung membentuk cluster bulat dengan ukuran sebanding; sensitif terhadap noise dan *outlier*; tanpa fitur posisi, region hasilnya **tidak dijamin terhubung** |

Berikut adalah code untuk penggunaan K-Means pada segmentasi citra.

```
import cv2
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Baca citra dan ubah ke RGB
filename = ('/content/drive/MyDrive/PCVK/Images/lily.jpg')

img = cv.imread(filename)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Ubah citra menjadi array 2D (piksel)
Z = img.reshape((-1, 3))
Z = np.float32(Z)

# Tentukan jumlah cluster
k = 3
kmeans = KMeans(n_clusters=k, random_state=42)
kmeans.fit(Z)

# Dapatkan label dan pusat cluster
labels = kmeans.labels_
centers = np.uint8(kmeans.cluster_centers_)
# Bentuk citra hasil segmentasi
segmented = centers[labels.flatten()]
segmented_img = segmented.reshape(img.shape)

# Tampilkan hasil
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(img)
plt.title('Citra Asli')
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(segmented_img)
plt.title('Segmentasi K-Means (k=3)')
plt.axis('off')
plt.show()
```

Berikut adalah hasil dari code diatas

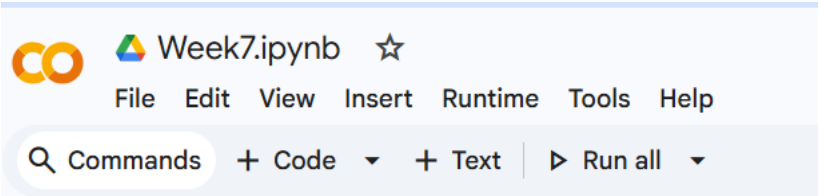



Gambar 18. Hasil segmentasi citra menggunakan K-Means Clustering

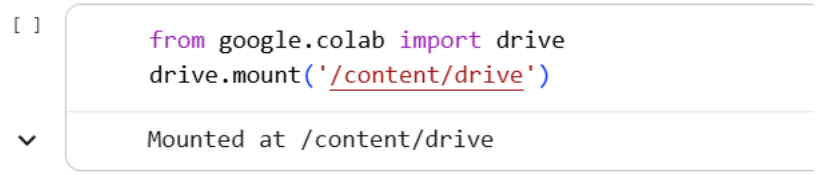
## D. TUGAS PRAKTIKUM

### D1. Tugas Individu

1. Buka https://colab.research.google.com/. Setelah dipastikan bahwa google Colab terhubung dengan Github Anda, lanjutkan dengan memilih repository yang telah digunakan pada praktikum minggu lalu, rename file menjadi “Week7.ipynb”.



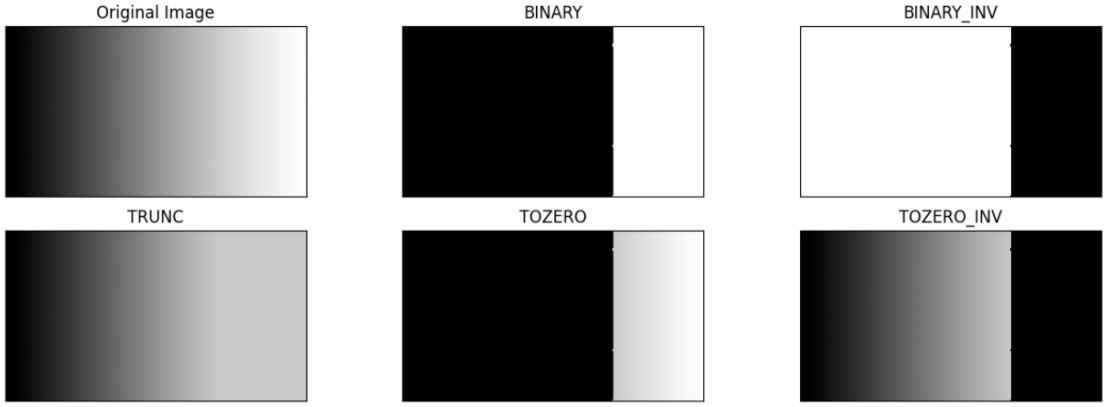

Kemudian import folder yang ada di Drive Anda dengan cara sebagai berikut




2. Import beberapa library berikut yang akan digunakan selama uji coba praktikum minggu ke-7 berikut.

```
#Library yang digunakan
import cv2 as cv
from google.colab.patches import cv2_imshow
import numpy as np
import math
import pandas as pd
import matplotlib.pyplot as plt
from collections import deque
from scipy import ndimage as ndi
from skimage import data, filters, color, segmentation, feature, morphology
from skimage.filters import gaussian
from skimage.segmentation import flood
```

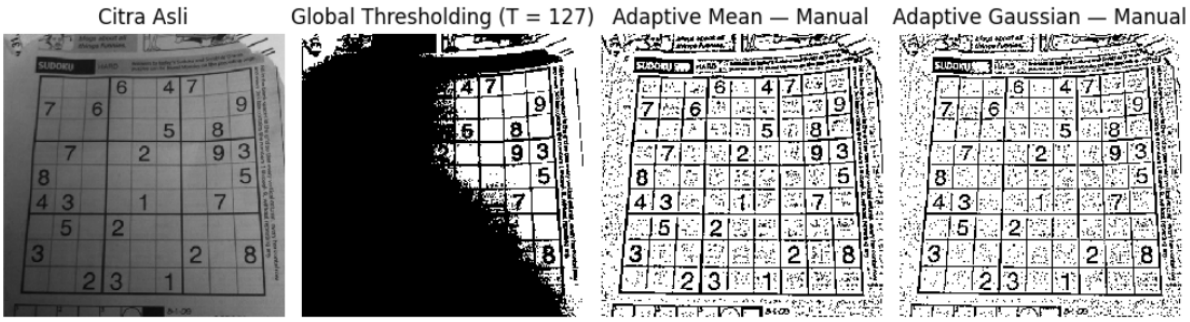
3. Buat Global Threshold (BINARY, BINARY_INV, TRUNC, TOZERO, TOZERO_INV), dengan threshold= 200, **secara manual** sehingga menghasilkan tampilan seperti berikut:




Ketentuan:

| Jenis Global Threshold | Jika nilai piksel > thresh | Jika nilai piksel ≤ thresh |
|---|---|---|
| **BINARY** | Menjadi nilai maksimal | Menjadi 0 |
| **BINARY_INV** | Menjadi 0 | Menjadi nilai maksimal |
| **TRUNC** | Menjadi thresh | Nilai asli tetap |
| **TOZERO** | Nilai asli tetap | Menjadi 0 |
| **TOZERO_INV** | Menjadi 0 | Nilai asli tetap |

4. Buat program Addaptive Threshold : adaptive mean dan adaptive Gaussian **tanpa menggunakan cv.threshold(), cv.adaptiveThreshold(), maupun fungsi filter siap pakai** dengan menggunakan **citra Sudoku** yang memiliki pencahayaan tidak merata. Ketentuan: global threshold secara manual dengan ambang **127, ukuran jendela 11 × 11, C=2, dan sigma=2 untuk Gaussian** sehingga menghasilkan tampilan seperti berikut:




5. A. Buat Otsu Thresholding tanpa menggunakan Library. Tampilkan juga nilai threshold saat anda gunakan Otsu's, seperti terlihat pada gambar hasil berikut. (gunakan image **lena_gs_lc2.jpg** untuk dibandingkan dengan hasil dengan penggunaan library. B. Kemudian coba dengan image **KTP** sendiri agar terlihat beda antara hasil otsu's dengan global threshold biasa)

```
filename = ('/content/drive/MyDrive/PCVK/Images/lena_gs_lc2.jpg')
img = cv.imread(filename,0)
blur = cv.GaussianBlur(img,(5,5),0)
thresh=127

def otsu(gray):
    pixel_number = gray.shape[0] * gray.shape[1]
    mean_weight = 1.0/pixel_number
    his, bins = np.histogram(gray, np.arange(0,257))
    final_thresh = -1
    final_value = -1
    intensity_arr = np.arange(256)
    for t in bins[1:-1]:
        pcb = np.sum(his[:t])
        pcf = np.sum(his[t:])
        Wb = pcb * mean_weight
        Wf = pcf * mean_weight

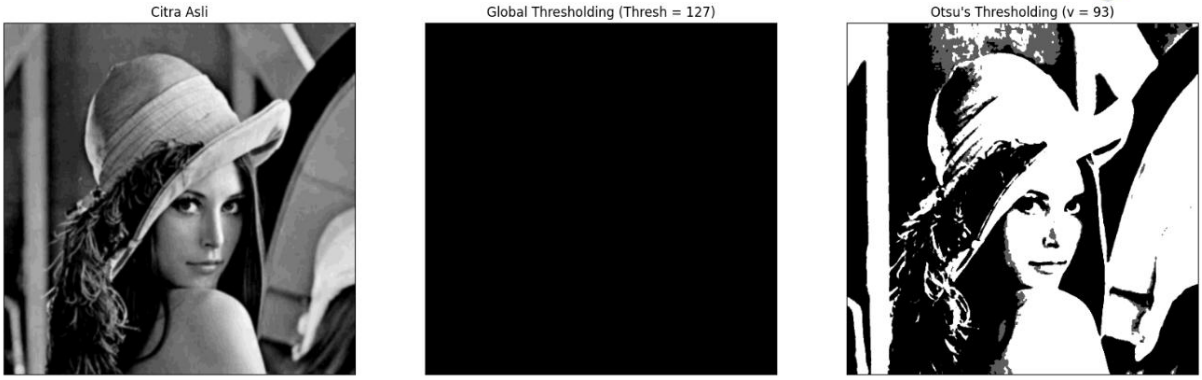
        mub = np.sum(intensity_arr[:t]*his[:t]) / float(pcb)
        muf = np.sum(intensity_arr[t:]*his[t:]) / float(pcf)
        value = Wb * Wf * (mub - muf) ** 2

        if value > final_value:
            final_thresh = t
            final_value = value
    final_img = gray.copy()
    print(final_thresh)
    final_img[gray > final_thresh] = 255
    final_img[gray < final_thresh] = 0
    return final_img, final_thresh

otsu_biner, otsu_thresh = otsu(blur)
x = ("Otsu's Thresholding (v = ")+str(otsu_thresh)+")"
ret,th1 = cv.threshold(blur,thresh,255,cv.THRESH_BINARY)

titles = ['Citra Asli', 'Global Thresholding (Thresh = ' + str(thresh) +
')', x]
citra3 = [blur, th1, otsu_biner]

plt.figure(figsize = (20,15))
for i in range(len(citra3)):
    plt.subplot(1,3,i+1),plt.imshow(citra3[i],'gray')
    plt.title(titles[i])
    plt.xticks([]),plt.yticks([])
plt.show()
```




**Pertanyaan**: Mengapa hasil tampilan Binary Thresholding semua bernilai 0 atau hitam semuan? Jelaskan alasannya dalam bentuk histogram dan berapa angka ambang batas (Thresh) supaya gambar Lena ini masih terlihat?

**Hasil dengan KTP_Fulan** (Bandingkan dengan hasil yang diperoleh)



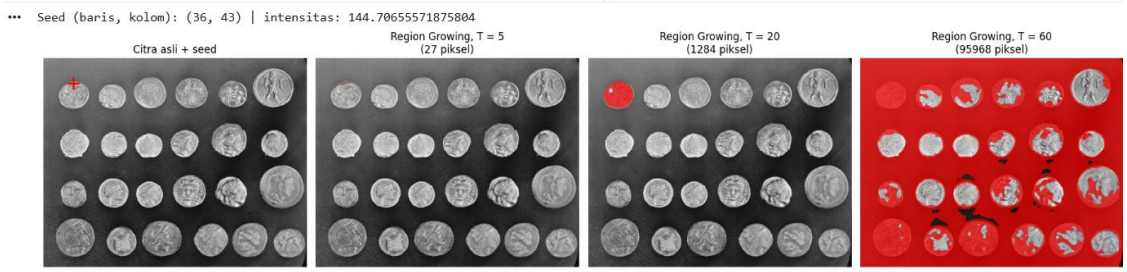

6. A. Lakukan Segmentasi Berbasis Region pada citra data.coins() dari skimage.

```python
from skimage import data
img = data.coins()          # grayscale, 303 x 384
```

Citra ini cocok karena memuat objek yang jelas (koin), latar yang cukup homogen, dan beberapa koin yang saling bersentuhan. Ketentuan:

- Tentukan 3 seed: satu di dalam koin, satu di latar, dan satu tepat di tepi koin.
- Jalankan dengan T = 5, 20, dan 60.
- Tampilkan hasilnya dalam grid 3×3 (seed × T)

Hasil yang diharapkan (Jumlah piksel belum tentu sama persis dengan contoh):




B. Gunakan **skimage.segmentation.flood()**. Fungsi ini menghasilkan mask region dari seed dengan menerima piksel yang terhubung dan intensitasnya berada dalam toleransi terhadap intensitas seed.

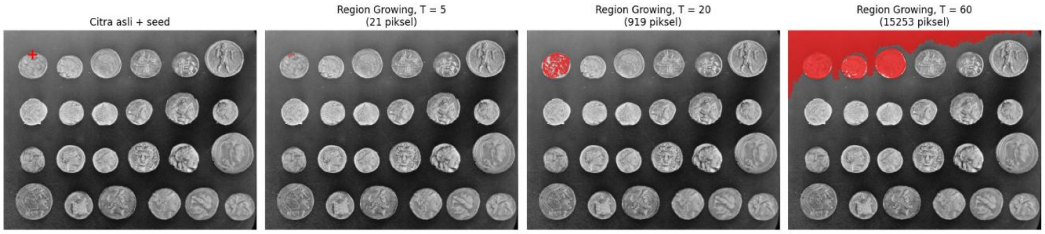
```python
import numpy as np
import matplotlib.pyplot as plt
from skimage import data
from skimage.segmentation import flood

gray = data.coins()
seed = (121, 63)          # (baris, kolom)

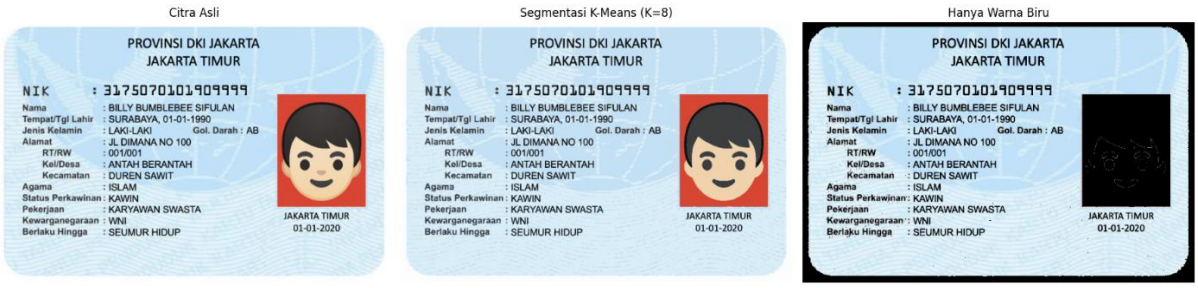
# Segmentasi menggunakan fungsi siap pakai
region = flood(
    gray,
    seed_point=seed,
    tolerance=20,
    connectivity=1       # 4 tetangga untuk citra 2D
)
```

Hasilnya kurang lebih seperti ini:

```
Seed (baris, kolom): (36, 43) | intensitas: 159.000000
Toleransi  5: 21 piksel
Toleransi 20: 919 piksel
Toleransi 60: 15253 piksel
```




7. Lakukan segmentasi warna pada image **ktp_fulan.jpg** , munculkan hanya warna yang **biru saja.** (Petunjuk: anda dapat gunakan K-Means untuk menampilkan hanya warna tertentu saja).
   Rentang mencakup biru muda kebiruan pada latar KTP (dengan warna HSV):
   kelompok_biru = np.where(  (H >= 85) & (H <= 130) &  (S >= 25) & (V >= 40) )




## D2. Tugas Kelompok

**Proses konversi gambar KTP menjadi teks yang dapat mengenali tulisan.**

Proses konversi gambar menjadi teks dikenal dengan istilah OCR **(*Optical Character Recognition*)** adalah teknologi untuk mengenali tulisan pada gambar dan mengubahnya menjadi teks digital yang dapat disalin, diedit, atau diolah.

**Tesseract** merupakan mesin OCR, sedangkan **pytesseract** digunakan untuk mengaksesnya melalui Python. Ketepatan pembacaan dipengaruhi oleh kejelasan tulisan, pencahayaan, ukuran huruf, dan latar belakang gambar. Langkah awal adalah melakukan Instal Tesseract dan model bahasa Indonesia.

```
!apt-get -qq update
!apt-get -qq install tesseract-ocr tesseract-ocr-ind
!pip -q install pytesseract
```

Latihan: **Lakukan Perbandingan Pembacaan Teks pada Citra Berwarna dan Hasil Segmentasi**

Gunakan satu citra **KTP masing-masing anggota** yang memuat tulisan dengan ukuran dan warna latar yang beragam.

**Instruksi pengerjaan:**

1. **Siapkan tiga citra untuk dibandingkan:**
   - Citra berwarna asli.
   - Citra hasil segmentasi menggunakan **Otsu Thresholding**.
   - Citra hasil segmentasi menggunakan **Adaptive Thresholding**.

   Gunakan ukuran citra yang sama. Untuk kedua metode thresholding, ubah citra menjadi grayscale terlebih dahulu.
2. **Lakukan pembacaan teks pada ketiga citra** menggunakan pytesseract.image_to_string(). Gunakan bahasa dan pengaturan --psm yang sama agar perbandingan konsisten.
3. **Tampilkan area teks yang dikenali** menggunakan pytesseract.image_to_data() dan kotak hijau. Gunakan batas confidence yang sama, misalnya **lebih dari 60**, pada ketiga citra.
4. **Bandingkan hasil pembacaan** dengan tulisan sebenarnya pada gambar. Pilih **10 kata atau angka** yang terlihat jelas sebagai acuan, lalu isi tabel berikut. Penilaian dilakukan pada teks hasil OCR, bukan hanya pada kata yang diberi kotak.

| No. | Teks sebenarnya | OCR citra berwarna | OCR Otsu | OCR Adaptive |
|---|---|---|---|---|
| 1 | … | … | … | … |
| 2 | … | … | … | … |
| … | … | … | … | … |
| 10 | … | … | … | … |

5. **Hitung persentase kecocokan** untuk setiap metode. Hasil dinyatakan cocok jika seluruh huruf atau angka pada item sama dengan acuan; abaikan perbedaan huruf besar dan kecil serta spasi di awal/akhir.

$$\text{Kecocokan} = \frac{\text{Jumlah item yang terbaca tepat}}{10} \times 100\%$$

Item yang tidak terbaca dihitung sebagai tidak cocok. Lengkapi ringkasan berikut:

| Citra masukan OCR | Item terbaca tepat | Kecocokan (%) | Rata-rata confidence |
|---|---|---|---|
| Berwarna asli | … | … | … |
| Segmentasi Otsu | … | … | … |
| Segmentasi Adaptive | … | … | … |

**Pertanyaan analisis:**

1. Citra masukan mana yang menghasilkan pembacaan paling tepat? Tunjukkan buktinya dari tabel.
2. Sebutkan minimal dua kesalahan pembacaan, seperti huruf tertukar, angka salah, atau teks tidak terbaca.
3. Apakah segmentasi selalu meningkatkan ketepatan OCR? Jelaskan berdasarkan hasil percobaan.
4. Bagaimana warna atau pola latar belakang memengaruhi pembacaan teks?
5. Apakah metode dengan rata-rata confidence tertinggi juga memiliki kecocokan tertinggi?

**Tiga citra untuk dibandingkan:**

Berwarna




Otsu




Adaptive




## Hasil Pembacaan Teks

Area Teks — Berwarna




Area Teks — Otsu




Area Teks — Adaptive




**Contoh Hasil Pembacaan Citra Berwarna:**

```
========== BERWARNA ==========
PROVINSI DKI JAKARTA
JAKARTA TIMUR

NIK : 31?50?01014041419
Nama : BILLY BUMBLEBEE SIFULAN
Tempat/Tgl Lahir : SURABAYA, 01-01-1990
Jenis Kelamin : LAKI-LAKI Gol. Darah : AB
Alamat : JL DIMANA NO 100

RT/RW : 001/001

Kel/Desa : ANTAH BERANTAH t /

Kecamatan : DUREN SAWIT
Agama : ISLAM
Status Perkawinan : KAWIN
Pekerjaan : KARYAWAN SWASTA
Kewarganegaraan : WNI JAKARTA TIMUR
Berlaku Hingga – : SEUMUR HIDUP 01-01-2020
```

**--- SELAMAT BELAJAR ---**